

Assignment 3

End-to-End Hugging Face Model Training & Docker Deployment

Jahanvi Gajera (M25CSA012)

February 25, 2026

1 Introduction

This project demonstrates a complete end-to-end Machine Learning workflow using the Hugging Face ecosystem. The objective of this assignment was to:

- Convert a Jupyter Notebook workflow into production-ready Python scripts
- Train and evaluate a transformer-based text classification model
- Containerize the workflow using Docker
- Push the trained model to Hugging Face Hub
- Re-evaluate the model from the Hugging Face repository
- Publish the complete project to GitHub

The task focuses on fine-tuning a transformer model for multi-class book genre classification.

2 Environment Setup Using Docker

To ensure reproducibility and portability, Docker was used.

2.1 Docker Configuration

Three Docker files were created:

- Dockerfile.train – Used for training pipeline
- Dockerfile.eval – Used for evaluation-only production image
- Dockerfile – General development setup

2.2 Base Image

`python:3.10-slim` was used as the base image because it is lightweight and production-friendly.

2.3 Installed Dependencies

- `transformers`
- `datasets`
- `torch`
- `scikit-learn`
- `pandas`
- `numpy`
- `huggingface_hub`

2.4 Docker Commands

```
docker build -t m25csa012_train -f Dockerfile.train .
```

```
docker run HF_TOKEN="**" m25csa012_eval python /src/train.py
```

```
docker build -t m25csa012_eval -f Dockerfile.eval .
```

```
docker run HF_TOKEN="**" m25csa012_eval python /src/evaluate.py
```

3 Notebook to Production Scripts

The notebook was converted into modular Python scripts structured as:

```
project/  
  data.py  
  train.py  
  evaluate.py  
  utils.py  
  requirements.txt  
  Dockerfile  
  Dockerfile.train  
  Dockerfile.eval
```

4 Model Selection from Hugging Face

The pre-trained model used:

4.1 Reason for Selection

- Lightweight and faster than BERT
- Suitable for text classification
- Lower computational cost
- Fully supported by Hugging Face Trainer API
- I set max_length=128 in data.py file for less computation because i used my system's CPU therefore eval accuracy is also very less.

5 Training Using Hugging Face Trainer API

5.1 Dataset

The dataset contains 8 balanced genres:

- Romance
- Mystery_Thriller_Crime
- Young_Adult
- History_Biography
- Comics_Graphic
- Fantasy_Paranormal
- Children
- Poetry

Each class contains 200 samples (Total = 1600 evaluation samples).

5.2 Training Configuration

- Optimizer: AdamW
- Loss Function: CrossEntropyLoss
- Evaluation Strategy: Per epoch
- Metrics: Accuracy and Weighted F1-score

6 Local Model Evaluation Results

6.1 Overall Metrics

Metric	Value
Eval Loss	3.2344
Accuracy	0.1444
Weighted F1	0.1441

Table 1: Local Evaluation Results

6.2 Observations

- Best performing class: History_Biography (F1 = 0.494)
- Low performance in Romance, Mystery, and Children
- Model shows signs of underfitting

7 Model Deployment to Hugging Face Hub

The model and tokenizer were pushed using:

```
model.push_to_hub("Jahanvi16/tinybert_model")
tokenizer.push_to_hub("Jahanvi16/tinybert_model")
```

The Hugging Face repository includes:

- pytorch_model.bin
- config.json
- tokenizer files
- training arguments

8 Re-Evaluation from Hugging Face Hub

8.1 Hugging Face Evaluation Metrics

Metric	Value
Eval Loss	2.8062
Accuracy	0.1288
Weighted F1	0.1175

Table 2: Evaluation Results from Hugging Face Repository

9 Comparison: Local vs Hugging Face Model

Metric	Local Model	HF Hub Model
Eval Loss	3.2344	2.8062
Accuracy	0.1444	0.1288
Weighted F1	0.1441	0.1175

Table 3: Performance Comparison

9.1 Analysis

- Slight accuracy drop after reloading from Hub
- Similar performance overall
- Minor differences due to environment or seed variations

10 Production Docker Image

A lightweight evaluation-only Docker image was created:

- Pulls model from Hugging Face
- Automatically runs evaluation
- Outputs JSON metrics

This ensures reproducibility and production readiness.

11 Reference links

- GitHub: https://github.com/jahanvi0106/MLOps-Jahanvi_Gajera-M25CSA012/tree/Assignment3
- Huggingface: [Jahanvi16/tinybert_model](#)

12 Key Learnings

- Hugging Face Trainer API usage
- Transformer fine-tuning
- Docker containerization
- Model deployment
- Reproducible ML pipelines

13 Limitations and Future Improvements

13.1 Current Limitations

- Low accuracy (14%)
- No hyperparameter tuning
- No early stopping

13.2 Future Improvements

- Increase training epochs
- Use learning rate scheduler
- Try larger transformer models
- Perform hyperparameter tuning
- Apply advanced preprocessing

14 Conclusion

This project successfully demonstrates a complete machine learning lifecycle:

Notebook → Scripts → Training → Evaluation → Hugging Face Deployment →
Dockerization → GitHub Publication

The pipeline is:

- Reproducible
- Containerized
- Publicly Deployable
- Production-Ready